

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 5

Komponenten und Standards

5.1 Allgemeines zu Komponenten

Die Art und Weise, PHP-Applikationen zu entwickeln, hat sich in den letzten Jahren stark verändert.

5.1.1 Von DIY zu Frameworks

In den Anfängen der Sprache war die Wiederverwendbarkeit von Code nur ein untergeordnetes Ziel. PHP wurde meist als Mittel zum Zweck eingesetzt, um dynamische Inhalte zu erzeugen. Lösungen für bestimmte Problemstellungen mussten selbst umgesetzt werden.

Etwa ab Mitte der 2000er Jahre entstanden die ersten PHP-Frameworks wie CakePHP und Symfony im Jahr 2005 oder CodeIgniter und das Zend Framework im Jahr 2006 [2]. Diese Frameworks boten erstmals ein großes Set an oft benötigten Funktionalitäten an, wie etwa Datenbankzugriffe, integrierte Template-Systeme, Session-Verwaltung u. v. m. Mit ihnen konnten nun Webprojekte zum ersten Mal in geordneteren Bahnen abgewickelt werden. Hinter den Frameworks versammelte sich schnell eine Community, die nicht nur für die Weiterentwicklung sondern auch am Support für Anwender*innen eine maßgebliche Rolle spielte. PHP-Entwickler*innen waren nun mit ihren Problemen nicht mehr alleine bzw. mussten diese selbst lösen, sondern konnten sich Unterstützung bei anderen Entwickler*innen holen, die ebenfalls Erfahrungen mit dem jeweiligen Framework hatten.

Frameworks bedeuten jedoch auch immer eine Art von geschlossenem Ökosystem, in das zum Erlernen viel Zeit investiert werden muss. Wird das Framework dann beherrscht, so erfordern andere Projekte möglicherweise andere Frameworks, was einen erneuten (wenn wohl auch verkürzten) Lernprozess zur Folge hat. Wird ein Framework für ein Webprojekt ausgewählt, so kommt dies auch einer Investition in dessen Zukunft gleich. Es gilt abzuschätzen, ob das Framework auch weiterentwickelt wird, um nicht Gefahr zu laufen, dass in naher Zukunft keine Updates mehr zur Verfügung gestellt werden.

Schließlich stellt sich auch immer die Frage, ob ein Framework überhaupt benötigt wird, da es, wie bereits erwähnt, eine Vielzahl von Funktionalitäten enthält, die oft nur bei großen Projekten Sinn machen. Beispiele hierfür können große öffentliche

Seiten oder Plattformen sein. Für kleinere Projekte, wie etwa private Anwendungen oder Webprojekte mit sehr spezifischen Anwendungsgebieten stellt ein PHP-Framework möglicherweise zu viel unnötigen Overhead dar.

Diesem Umstand kann seit einigen Jahren mit einem neuen Konzept begegnet werden: Komponenten.

5.1.2 PHP-Komponenten

Eine *Komponente* ist eine Sammlung von PHP-Code, d. h. zusammengehörige Klassen, Interfaces und Traits – die eine spezifische Aufgabe erfüllt bzw. ein Problem löst. Im Unterschied zu Frameworks ist dies genau *eine* Aufgabe. So existieren etwa Komponenten, um E-Mails zu versenden, HTTP-Requests zu tätigen oder Logging für Webapplikationen zur Verfügung zu stellen. Gute PHP-Komponenten sollten dabei die folgenden Kriterien erfüllen:

- **Fokussiert auf ein Problem:** Eine gute Komponente existiert, um genau ein Problem zu lösen bzw. eine Aufgabe zu erfüllen. Dies ermöglicht es, Overhead und unnötigen Code in der eigenen Applikation zu vermeiden, da durch die Verwendung einer Komponente nicht unnötige Funktionalitäten „eingekauft“ werden.
- **Klein:** PHP-Komponenten sind nicht größer, als sie sein müssen. Sie enthalten nur so viel Code wie notwendig, um ihre Aufgabe zu erfüllen. Diese Größe kann natürlich variieren. So gibt es Komponenten, die nur aus einer Klasse bestehen, während andere eine Sammlung verschiedener Klassen darstellen.
- **Kooperativ:** PHP-Komponenten arbeiten gut mit anderen Komponenten zusammen bzw. behindern diese nicht. So sollten Komponenten ihren eigenen Namespace verwenden, um Namenskollisionen zu vermeiden. Weiters können Komponenten auf anderen aufbauen, wenn Funktionalitäten benötigt werden, die schon existieren.
- **Gut getestet:** Komponenten sollten gut getestet sein, um potentielle Probleme zu minimieren. Durch ihren Fokus und ihre Kleinheit sind Tests in der Regel nicht umfangreich. Gute Komponenten inkludieren diese Tests, sodass sie von anderen Entwickler*innen ausgeführt und nachvollzogen werden können. Mehr zu Tests in Vorlesung 12.
- **Gut dokumentiert:** Um die Verwendung zu erleichtern, sollten Komponenten über eine gute Dokumentation verfügen. Eine README-Datei erklärt, was die Komponente tut, wie sie installiert werden kann und wie sie eingesetzt wird. Kommentare im Code (so genannte Docblocks) erläutern Klassen und Methoden näher, daraus kann in wenigen Schritten eine webbasierte API-Dokumentation generiert werden.

Eine moderne PHP-Webapplikation muss also nicht zwingend auf ein großes Framework setzen (wenngleich moderne Frameworks wie Symfony¹, Laravel² oder Yii³ wie erwähnt für größere Projekte absolut empfehlenswert sind). Vielmehr sollte zunächst genau überlegt und analysiert werden, welche Funktionalitäten benötigt werden. Da-

¹<https://symfony.com/>

²<https://laravel.com/>

³<https://www.yiiframework.com/>

nach kann recherchiert werden, ob es für diese Aufgaben bereits Komponenten gibt und falls ja, werden diese in das eigene Projekt integriert und verwendet. Somit muss das Rad an mehreren Stellen nicht neu erfunden werden (d. h. Probleme gelöst werden, für die es bereits etablierte Lösungen gibt) und der Overhead und die Einlernphase für ein großes Framework entfallen ebenfalls.

5.2 Komponenten verwalten: Composer & Packagist

Für die Installation und Verwaltung von Komponenten hat sich in PHP eine Gesamtlösung etabliert: Das Tool *Composer* in Kombination mit dem Komponentenverzeichnis *Packagist*.

5.2.1 Composer

*Composer*⁴ ist ein sogenannter Dependency-Manager, manchmal auch Paketmanager genannt. Seine Aufgabe besteht darin, Komponenten für ein Projekt zu installieren und deren Abhängigkeiten (andere Komponenten, die benötigt werden) aufzulösen und ebenfalls herunterzuladen (und ggf. wiederum deren Abhängigkeiten usw.). Die Komponenten – auch Packages genannt – werden dabei auf Projekt-Basis installiert (nicht systemweit). Somit ist Composer nicht mit klassischen Paketmanagern wie Apt unter Linux vergleichbar, die Pakete für das gesamte System installieren. Composer hat jedoch eine Option, Pakete global zu installieren, etwa wenn es sich um Tools (Binaries) handelt, die in mehreren Projekten verwendet werden können. Die aus dem ersten Semester schon bekannte Entsprechung in JavaScript ist *npm*⁵.

Darüber hinaus kümmert sich Composer auch um das Autoloading der installierten Komponenten, d. h. es wird eine zentrale Autoloading-Datei erstellt (*autoload.php*), die in der Hauptdatei des eigenen Projekts eingebunden wird und sich um die Einbindung aller Komponenten selbst kümmert.

Composer ist eine Commandline-Applikation, die auf dem Computer, auf dem der Webserver läuft, d. h. in der Regel ein entfernter Server, genauso aber ein Docker-Container oder der eigene Computer mit einer lokalen PHP-Installation, installiert wird. Die aktuelle Version ist 2.9.7.

Composer installieren

Composer kann lokal im Projekt oder global im System installiert werden. Die ersten vier Schritte sind dabei identisch. Unter Linux, Unix bzw. macOS lauten sie:

```
1 php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
2 php -r "if (hash_file('sha384', 'composer-setup.php') === '
    c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eaaf77b3419b0bf2768dc67c86944d
    a1544f06fa544fd47') { echo 'Installer verified'.PHP_EOL; } else { echo '
    Installer corrupt'.PHP_EOL; unlink('composer-setup.php'); exit(1); }"
3 php composer-setup.php
4 php -r "unlink('composer-setup.php');"
```

⁴<https://getcomposer.org/>

⁵<https://www.npmjs.com/>

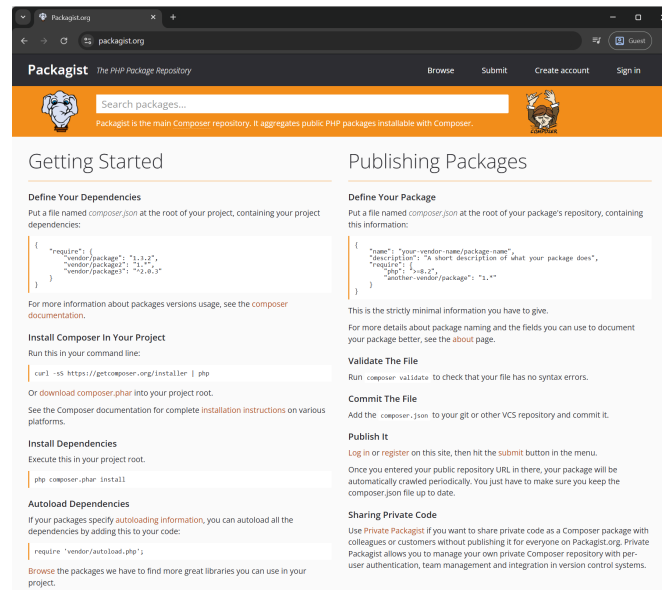


Abbildung 5.1: Die Startseite von Packagist. In der Suchzeile kann nach Paketen gesucht werden, darunter werden Informationen zur Verwendung mit Composer angezeigt.

Dies lädt zunächst den aktuellen Composer-Installer herunter und benennt ihn in `composer-setup.php` um. Danach wird die SHA384-Checksumme überprüft, um sicherzustellen, dass der Installer korrekt heruntergeladen wurde. Falls dies der Fall ist, wird `composer-setup.php` ausgeführt und anschließend gelöscht. Das Ergebnis ist die Datei `composer.phar` (PHAR steht für PHP Archive), die mit `php composer.phar` ausgeführt werden kann.

In der Regel wird Composer jedoch systemweit installiert. Dies wird dadurch erreicht, dass die PHAR-Datei nach `usr/local/bin/composer` verschoben wird. Dadurch ist sie im gesamten System mit dem Befehl `composer` aufrufbar (so ist dies etwa im *fhoee-web-dock* Docker-Container der Fall):

```
1 mv composer.phar /usr/local/bin/composer
```

Um Composer für Windows zu installieren, wird der Installer `Composer-Setup.exe` von <https://getcomposer.org/download/> heruntergeladen und ausgeführt. Dies installiert alles notwendige und ändert die Pfadvariablen, sodass der Befehl `composer` von überall her ausgeführt werden kann.

5.2.2 Packagist Repository

Das zentrale Verzeichnis für PHP-Komponenten ist *Packagist*⁶ (siehe Abbildung 5.1). In ihm werden öffentlich verfügbare Komponenten aggregiert und über Schlüsselworte und Packagenamen durchsuchbar gemacht. Packagist listet dabei verfügbare Komponenten auf und gibt dabei auch aus, wie viele Downloads die Komponente aufweist bzw. wie oft sie von Nutzer*innen mit Sternen versehen wurde (um sie positiv hervorzuheben).

⁶<https://packagist.org/>

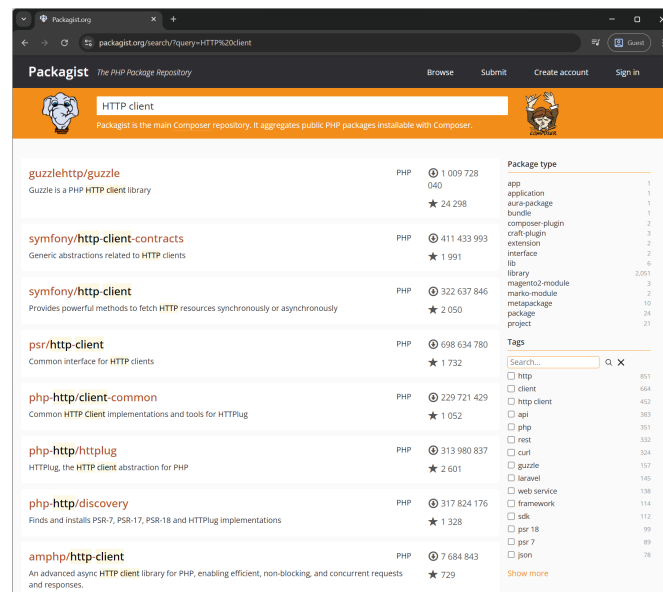


Abbildung 5.2: Auf Packagist wird eine PHP-Komponente für HTTP-Requests gesucht. Es werden mehrere Ergebnisse angezeigt.

Die Anzahl der auf Packagist geführten PHP-Komponenten ist sehr hoch. Mit Stand Mai 2026 sind es 451.631, die in insgesamt 5.532.688 einzelnen Versionen vorliegen [4]. Um einen Überblick über die populärsten Komponenten zu bekommen, kann auf die Downloadzahlen zurückgegriffen werden. Diese sind jedoch nicht immer aussagekräftig, darum empfiehlt sich die Auflistung *Awesome PHP* [6], die eine kuratierte Liste an PHP-Komponenten darstellt.

Packages finden

Um nun eine Komponente zu installieren, wird diese zunächst über Packagist gesucht und dann mittels Composer im eigenen Projekt installiert.

Als Beispiel soll eine kleine Anwendung geschrieben werden, die Anfragen an die öffentliche Chuck Norris Jokes API⁷ sendet, um einen Fakt (Witz) über Chuck Norris abzufragen. Eine API ist eine Schnittstelle, die auf GET- oder POST-Anfragen hört und daraufhin Daten in einem maschinenlesbaren Format – meist JSON – zurückliefert. Mehr zu APIs folgt in Vorlesung 11.

Als erstes wird eine Komponente gesucht, die HTTP-Requests verarbeiten kann, da die Chuck Norris Jokes API den Fakt (d. h. die Daten) als Response auf einen GET-Request liefert. Dazu wird auf Packagist „HTTP client“ als Suchbegriff verwendet (siehe dazu Abbildung 5.2).

Eines der ersten Resultate ist **guzzlehttp/guzzle**: „Guzzle is a PHP HTTP client library“. Dies klingt vielversprechend und die hohe Anzahl der Downloads lässt ebenfalls darauf schließen, dass es sich um eine solide Komponente handelt.

⁷<https://api.chucknorris.io/>

Komponentennamen

Am Beispiel der Komponente `guzzlehttp/guzzle` zeigt sich das Namensschema der Komponenten auf Packagist. Dieses lautet immer `vendor/package`. Jede volle Packagebezeichnung besteht aus einem Vornamen – hier `guzzlehttp` – und einem Packagenamen – hier `guzzle`. Der Vorname ist global einzigartig und ist entweder der Name eines Projekts oder einer Firma bzw. Organisation, die diese Komponente (und auch andere) entwickelt. Unter dem Vornamen `guzzlehttp` finden sich etwa 18 verschiedene Packages. Der Packagename selbst bezeichnet die Komponente, die installiert werden soll. Für die Installation des Packages sind beide Teile notwendig.

Komponenten installieren

Mit dem Wissen, dass die benötigte HTTP-Client-Komponente unter dem vollen Namen `guzzlehttp/guzzle` zu finden ist, kann nun Composer verwendet werden, um sie zu installieren.

Die Syntax für eine Installation mit Composer in der Kommandozeile lautet wie folgt:

```
1 $ composer require vendor/package
```

`composer` ruft die Composer-Executable auf, mit dem Schlüsselwort `require` wird eine benötigte Komponente angefordert. Es folgt der volle Name aus Vendor- und Packagenamen.

Für das konkrete Beispiel wird in der Kommandozeile (im Docker Container mittels `docker exec -it webapp /bin/bash`) zunächst ein Projektordner `chucknorrisfact` angelegt, dann in diesen gewechselt und schließlich der Composer Executable mitgeteilt, dass das Package `guzzlehttp/guzzle` benötigt wird:

```
1 $ mkdir chucknorrisfact
2 $ cd chucknorrisfact
3 $ composer require guzzlehttp/guzzle
```

Dies startet den Installationsvorgang (siehe Abbildung 5.3) und erzeugt zunächst zwei Dateien im Ordner `chucknorrisfact`: `composer.json` und `composer.lock`. In die erstgenannte Datei werden die benötigten Komponenten im JSON-Format für die aktuelle Anwendung notiert. Die Datei kann entweder von Composer selbst (über den eben erfolgten `composer require`-Aufruf) modifiziert oder auch händisch bearbeitet werden. Der Inhalt nach dem initialen Aufruf sieht wie folgt aus:

```
1 {
2   "require": {
3     "guzzlehttp/guzzle": "^7.10"
4   }
5 }
```

Die Datei `composer.lock` ist eine Ergänzung zu `composer.json` und speichert die genauen Versionen, die installiert wurden. Dies hilft dabei, dass mehrere Personen, die an dem Projekt arbeiten, dieselben Versionen der Komponenten installieren, auch wenn es inzwischen neuere gibt. Beide Dateien sollten bei einer Arbeit mit Versionskontrolle (z. B. Git) unbedingt mitgespeichert werden, da sie zu einer späteren Installation der benötigten Komponenten unerlässlich sind.

```

root@webapp:/var/www/html/net2vo-examples/vo65# cd chucknorrisfact#
root@webapp:/var/www/html/net2vo-examples/vo65/chucknorrisfact# composer require guzzlehttp/guzzle
./composer.json has been created
Running composer update guzzlehttp/guzzle
Loading composer repositories with package information
Updating dependencies
Lock file operations: 8 installs, 0 updates, 0 removals
- Locking guzzlehttp/guzzle (7.10.0)
- Locking guzzlehttp/promises (2.3.0)
- Locking guzzlehttp/psr7 (2.9.0)
- Locking psr/http-client (1.0.3)
- Locking psr/http-factory (1.1.0)
- Locking psr/http-message (2.0)
- Locking ralouphie/getallheaders (3.0.3)
- Locking symfony/deprecation-contracts (v3.6.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 8 installs, 0 updates, 0 removals
- Downloading symfony/deprecation-contracts (v3.6.0)
- Downloading psr/http-message (2.0)
- Downloading psr/http-client (1.0.3)
- Downloading ralouphie/getallheaders (3.0.3)
- Downloading psr/http-factory (1.1.0)
- Downloading guzzlehttp/psr7 (2.9.0)
- Downloading guzzlehttp/promises (2.3.0)
- Downloading guzzlehttp/guzzle (7.10.0)
- Installing symfony/deprecation-contracts (v3.6.0): Extracting archive
- Installing psr/http-message (2.0): Extracting archive
- Installing psr/http-client (1.0.3): Extracting archive
- Installing ralouphie/getallheaders (3.0.3): Extracting archive
- Installing psr/http-factory (1.1.0): Extracting archive
- Installing guzzlehttp/psr7 (2.9.0): Extracting archive
- Installing guzzlehttp/promises (2.3.0): Extracting archive
- Installing guzzlehttp/guzzle (7.10.0): Extracting archive
3 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
4 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
No security vulnerability advisories found.
Using version ^7.10 for guzzlehttp/guzzle
root@webapp:/var/www/html/net2vo-examples/vo65/chucknorrisfact#

```

Abbildung 5.3: Composer installiert das Paket `guzzlehttp/guzzle` inklusive seiner Abhängigkeiten.

Neben den beiden Dateien wurde auch ein Ordner `vendor` angelegt, in den Composer die Dateien der Komponente, weitere Komponenten auf denen die eingebundene aufbaut und eigene Autoloading-Dateien abgelegt hat. Dieser Ordner kann sehr groß werden, es sollte daher auch nicht unter Versionskontrolle gestellt oder weitergegeben werden. Seine Inhalte können von Composer mit einem einfachen Aufruf sofort wieder generiert werden.

Existieren `composer.json` und `composer.lock` für ein Projekt nämlich bereits, so sind die benötigten Komponenten bereits bekannt und es reicht der folgende Aufruf, um diese zu installieren:

```
1 $ composer install
```

Versionsangaben

Wie ersichtlich ist, notiert Composer neben dem Packagenamen auch eine Versionsnummer, in diesem Fall `~7.10`. Dies gibt an, dass minimal Version 7.10 installiert werden, zudem werden auch kompatible, größere Updates installiert. Diese Syntax verlässt sich auf das sogenannte *Semantic Versioning*⁸, das Versionsnummern dreistellig im Format `MAJOR.MINOR.PATCH`, also z. B. 1.2.4 oder 3.0.8 angibt. Die Änderung der Versionsnummern folgt dabei genauen Regeln:

1. MAJOR ändert sich nur, wenn inkompatible, d. h. sehr fundamentale Änderungen an den Methoden (der API) durchgeführt werden.
2. MINOR ändert sich, wenn neue Funktionalitäten hinzugefügt werden, die jedoch rückwärtskompatibel sind.
3. PATCH ändert sich, wenn Fehler behoben werden, die auch rückwärtskompatibel sind.

⁸<https://semver.org/>

Da sich alle Komponenten, die auf Packagist veröffentlicht wurden, an Semantic Versioning halten (sollten), kann mit einer Versionsangabe genau gesteuert werden, welche Versionen Composer installieren darf. Die Versionsangabe folgt dem Packagenamen mit einem Doppelpunkt:

```
1 $ composer require vendor/package:^1.0
```

Die folgenden Notationen sind dabei möglich:

- Genaue Versionsangabe, z. B. 1.2.3: Installiert nur genau diese Version.
- Bereichsangaben, z. B. >=1.0 oder >= 1.0 <2.0: Mit Hilfe der Vergleichsoperatoren >, >=, <, <= und != können Reichweiten angegeben werden. Ein Leerzeichen dient als logische UND-Verknüpfung, mit Hilfe des ||-Zeichens kann ein logisches ODER angegeben werden.
- Bereichsangaben mit Bindestrich, z. B. 1.0 - 2.0: Entspricht dabei einer Angabe von >=1.0.0 <=2.0.0 schließt also die beiden Angaben mit ein.
- Wildcard-Angabe, z. B. 1.0.*: Ermöglicht jede beliebige Versionsnummer an der PATCH-Stelle, entspricht also >=1.0.0 <1.1.
- Nächstmögliches Release, z. B. ~1.2: Installiert immer die neueste Version auf der aktuell angegebenen Semantic Versioning Ebene. Das angegebene Beispiel entspricht also >=1.2.0 <2.0.0, während ~1.2.3 >=1.2.3 <1.3.0 entspricht.
- Nächstmögliches Release (großzügiger), z. B. ^1.2.3: Verhält sich ähnlich wie der Tilde-Operator, ist jedoch großzügiger, sodass alle rückwärtskompatiblen Updates installiert werden. Das angegebene Beispiel entspricht in diesem Fall >=1.2.3 <2.0.0, da neue MINOR-Versionen keine Kompatibilitätsprobleme erzeugen (sollten). Dieser Operator sollte wann immer möglich zum Einsatz kommen.

5.2.3 Packages updaten

Um nach einiger Zeit die eingebundenen Komponenten auf den neuesten Stand zu bringen, kann folgender Befehl (im Projektverzeichnis, also z. B. in `chucknorrisfact`) ausgeführt werden:

```
1 $ composer update
```

Dabei werden die neuesten Versionen der eingebundenen Komponenten – sofern vorhanden – heruntergeladen und im Projekt eingebunden, `composer.lock` wird aktualisiert. Dabei werden jedoch immer die Versionsauflagen aus der `composer.json`-Datei beachtet. Gibt es neue Versionen einer Komponente – etwa Version 8.0.0 von `guzzlehttp/guzzle` – so muss zunächst die Versionsnummer in `composer.json` entsprechend angepasst werden, damit ein Update möglich ist.

5.2.4 Autoloading

Nachdem Komponenten mittels Composer installiert wurde, bleibt die Frage, wie diese eingebunden werden. Während Composer die notwendigen Komponenten herunterlädt, wird auch ein Autoloader generiert, der direkt im `vendor` Verzeichnis abgelegt wird. Somit muss nur diese Datei eingebunden werden, somit sind alle Komponenten verfügbar.

```
1 require "vendor/autoload.php";
```

5.2.5 Beispielprojekt: Custom Chuck Norris Fact

Mit der heruntergeladenen HTTP-Komponente und dem Wissen über das einfach Einbinden des Autoloaders kann nun das Chuck Norris-Projekt umgesetzt werden.

Auf der Webseite der API⁹ steht ihre Verwendung. Es können GET-Requests an mehrere URLs geschickt werden, um verschiedene Ergebnisse zu erhalten:

- GET <https://api.chucknorris.io/jokes/random>: Liefert einen zufälligen Fakt.
- GET <https://api.chucknorris.io/jokes/random?category={category}>: Liefert einen zufälligen Fakt aus einer übergebenen Kategorie.
- GET <https://api.chucknorris.io/jokes/categories>: Liefert alle verfügbaren Kategorien zurück.

Das Ergebnis ist immer eine JSON-Datenstruktur. Da es sich um GET-Requests handelt, können die URL-Aufrufe direkt in die Browserzeile kopiert werden und das Ergebnis ist sichtbar.

Für das Projekt wird nun eine Datei `index.php` angelegt, die – der Einfachheit halber – in ihrem Ablauf prozedural gehalten wird.

Am Beginn der Datei steht ein PHP-Block, der zunächst die verfügbaren Kategorien abfragen muss, damit diese dann in einem HTML-Formular angezeigt werden können.

Zunächst wird der Autoloader aus dem `vendor`-Verzeichnis eingebunden, damit die mit Composer installierte Komponente direkt verwendet werden kann. Dann wird der Guzzle HTTP-Client angelegt, indem ein Objekt der Klasse `GuzzleHttp\Client` erstellt wird. Das entstehende Objekt, gespeichert in der Variable `$client` hat eine Methode `request($method, $uri, $options)`, die einen Request absenden kann. Der erste Parameter bestimmt die Art (hier GET), der zweite den URL, der aufgerufen werden soll. Im dritten Parameter können Optionen als Array angegeben werden. Wird ein Eintrag `query` angegeben, so können GET-Parameter mitübergeben werden. Das Ergebnis wird in einem Response-Objekt (hier `$categoriesResponse`) gespeichert.

```
1 require "vendor/autoload.php";
2
3 $client = new GuzzleHttp\Client();
4
5 $categoriesResponse = $client->request(
6     "GET",
7     "https://api.chucknorris.io/jokes/categories",
8 );
```

Die API liefert ihre Daten im JSON Format. Um diese Antwort abzufragen, wird mit der Methode `getBody()` der Inhalt der Response entnommen und mit Hilfe von `json_decode()` in eine PHP-Datenstruktur umgewandelt. In diesem Fall handelt es sich um ein Array, da auch in JSON ein Array enthalten ist. `$categories` ist also ein PHP-Array in dem alle Kategorien enthalten sind.

```
1 $categoriesBody = $categoriesResponse->getBody();
2 $categories = json_decode($categoriesBody);
```

Nun folgt in der Datei ein HTML-Formular, das mit dem Kategorien-Array eine Dropdown-Liste (`<select>`) erstellt und die Kategorien darin anzeigt. Die Dropdown-Liste hat den Namen `category`, die einzelnen `<option>`-Elemente erhalten den Namen

⁹<https://api.chucknorris.io/>

der Kategorie als `value`-Attribut – dieser Wert wird beim Abschicken übertragen und steht, da das Formular mit POST abgeschickt wird, somit in `$_POST["category"]`. Das Formular ruft dabei dieselbe Seite erneut auf, denn in `$_SERVER["SCRIPT_NAME"]` steht immer der Pfad zur aktuellen Datei.

```

1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Random Chuck Norris Fact</title>
5     <meta charset="UTF-8">
6   </head>
7   <body>
8     <h1>Get a Random Chuck Norris Fact</h1>
9     <form action="<?= $_SERVER["SCRIPT_NAME"] ?>" method="post">
10      <label for="categorySelector">Select a category for a fact:</label>
11      <select name="category" id="categorySelector">
12        <?php
13          foreach ($categories as $category) {
14            echo "<option value=\"\$category\">\"$category\"</option>";
15          }
16        ?>
17      </select>
18      <button type="submit">Roundhouse!</button>
19    </form>

```

Bevor `<body>` und `<html>` wieder geschlossen werden, wird nun die mögliche Ausgabe des zufälligen Chuck Norris Fakts vorgenommen.

Diese soll nur erfolgen, wenn in `$_POST["category"]` ein Wert enthalten ist, was nur der Fall ist, wenn das Formular auch abgeschickt wurde (nicht beim initialen Aufruf). Dann wird erneut das Guzzle-Client-Objekt `$client` verwendet, um einen GET-Request abzuschicken. Dieses Mal wird ein zufälliger Fakt abgefragt und als Query-Parameter wird die ausgewählte Kategorie mitgegeben. War die Kategorie etwa „animal“, entspricht dies folgendem Aufruf: <https://api.chucknorris.io/jokes/random?category=animal>.

Dann wird wieder der Body der Response ausgelesen und mit `json_decode()` in eine PHP-Datenstruktur umgewandelt. Dieses Mal entsteht ein Objekt in dessen Eigenschaft `value` der Fakt enthalten ist. Auf diesen wird bei der Ausgabe mit `echo` zugegriffen.

```

1 if (isset($_POST["category"])) {
2   echo "<h2>Random fact from category {$_POST["category"]}</h2>";
3
4   $factResponse = $client->request(
5     "GET",
6     "https://api.chucknorris.io/jokes/random",
7     [
8       "query" => [
9         "category" => $_POST["category"],
10      ],
11    ],
12  );
13
14  $factBody = $factResponse->getBody();
15  $fact = json_decode($factBody);
16  echo "<p>$fact->value</p>";
17 }

```

Abbildung 5.4 zeigt einen zufälligen Chuck Norris Fakt aus der Kategorie „science“.

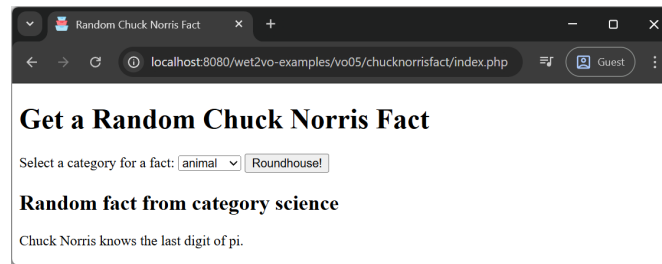


Abbildung 5.4: Ein Chuck Norris Fakt aus der Kategorie „science“. Die Dropdown-Liste zeigt nach dem Absenden des Formulars immer den ersten Eintrag in der Kategorienliste „animal“.

5.2.6 Eigene Komponenten definieren

Für das gezeigte Beispielprojekt wurde für die Umsetzung auf bestehende Komponenten zurückgegriffen. Es ist jedoch natürlich genauso möglich, eine eigene Komponente zu erstellen und diese über Packagist anderen zur Verfügung zu stellen. Dazu muss `composer.json` mehr Informationen enthalten als die bloße Angabe der verwendeten Dependencies. So muss etwa ein gültiger Packagename im Format `vendor/package` angegeben werden, sinnvoll sind auch eine Beschreibung, der Typ (z. B. `library` im Falle einer Bibliothek oder `project` im Falle eines gesamten Projekts), die Angabe einer Lizenz und der Autor*innen. Wenn Dependencies nötig sind, werden diese in gewohnter Form eingebunden.

Um eine initiale Version der `composer.json` zu erstellen, kann der folgende Befehl ausgeführt werden.

```
1 $ composer init
```

Dies startet einen kleinen interaktiven Assistenten, der die obigen Informationen abfragt und daraus die Datei generiert. Weitere Einträge können gemäß der Composer Dokumentation für das `composer.json`-Format [1] vorgenommen werden.

Der folgende Code zeigt `composer.json` für das `fhoee/router` Paket in Version 3.0.0, welches in dieser Lehrveranstaltung noch öfters zum Einsatz kommen wird:

```
1 {
2   "name": "fhoee/router",
3   "description": "A simple object-oriented router for educational purposes.",
4   "license": "MIT",
5   "type": "library",
6   "keywords": [
7     "routing",
8     "php",
9     "education"
10  ],
11  "authors": [
12    {
13      "name": "Wolfgang Hochleitner",
14      "email": "wolfgang.hochleitner@fh-hagenberg.at",
15      "role": "Developer"
16    }
17  ],
```

```

18  "require": {
19      "php": "^8.5",
20      "ext-mbstring": "*",
21      "psr/log": "^3.0"
22  },
23  "require-dev": {
24      "ergebnis/composer-normalize": "^2.50",
25      "friendsofphp/php-cs-fixer": "^3.94",
26      "mockery/mockery": "^1.6",
27      "pestphp/pest": "^4.4",
28      "phpstan/phpstan": "^2.1"
29  },
30  "minimum-stability": "stable",
31  "autoload": {
32      "psr-4": {
33          Fhooe\\Router\\: "src/"
34      }
35  },
36  "config": {
37      "allow-plugins": {
38          "ergebnis/composer-normalize": true,
39          "pestphp/pest-plugin": true
40      }
41  },
42  "scripts": {
43      "cs-check": "php-cs-fixer fix --dry-run --diff",
44      "cs-fix": "php-cs-fixer fix",
45      "pest": "pest",
46      "phpstan": "phpstan analyse src --memory-limit=-1 --level 9 || true",
47      "test": [
48          "@cs-check",
49          "@phpstan",
50          "@pest"
51      ]
52  }
53 }

```

5.3 Standards in PHP: PHP-FIG

Neben der Arbeit mit Komponenten gehört auch das Einhalten von Standards mittlerweile zum modernen Arbeitsstil in PHP.

5.3.1 Allgemeines zu Standards

Lange Zeit gab es für die Arbeit mit PHP nicht wirklich verbindliche Standards. Jedes Framework, jede*r Entwickler*in verfasste PHP-Code nach bestem Wissen und Gewissen bzw. nach eigenen Vorstellungen und Richtlinien. Dies führte zu einem Kommunikationsproblem. Code aus einem Framework konnte kaum in anderen Frameworks verwendet werden, da im Bezug auf Coding Standards Uneinigkeit herrschte.

Dieses Problem wurde von Entwickler*innen erkannt und ab 2009 mit Gründung der PHP Framework Interopability Group, kurz PHP-FIG¹⁰ aktiv bekämpft. Diese Or-

¹⁰<https://www.php-fig.org/>

ganisation hat es sich zum Ziel gesetzt, De-facto Standards zu entwickeln, die Zusammenarbeit zwischen verschiedenen Projekten erleichtern bzw. überhaupt ermöglichen.

Diese De-facto Standards werden *PSR* (PHP Standard Recommendation) und *PER* (PHP Evolving Recommendation) genannt. Sie sind in keinsten Weise verbindlich, haben sich aber in kürzester Zeit innerhalb der PHP-Community etabliert und erfreuen sich daher weiter Verbreitung. Im Gegensatz zu anderen Sprachen (etwa JavaScript) sind diese Standards auch die einzig akzeptierten. PSRs sind nummeriert¹¹, zur Zeit gibt es 14 akzeptierte, vier im Entwurfsstatus, drei eingestellte und zwei überholte.

PSRs lassen sich grob in drei Kategorien einteilen:

- **Styles:** Code-Richtlinien, die für die einheitliche Formatierung von PHP-Code in den verschiedensten Projekten sorgen sollen.
- **Autoloading:** Definierte Autoloading-Strategien, die es ermöglichen, mehrere Komponenten mit einem einzigen Autoloader zu laden.
- **Interfaces:** Standardisierte Schnittstellen, für immer wiederkehrende Aufgaben (wie z. B. Logging).

Neben den vielen PSRs existiert seit Juni 2022 auch der erste *PER*¹² (kurz für PHP Evolving Recommendation). Diese wurden eingeführt, um auf die kontinuierliche Weiterentwicklung der Sprache PHP besser reagieren zu können und betrifft vor allem Coding-Richtlinien (Styles). Denn ein PSR durchläuft einen langwierigen Prozess, bis er verabschiedet wird und stellt den Status zu genau einem Zeitpunkt da. Eine Coding-Guideline muss sich aber mit der Sprache weiterentwickeln, weshalb diese nun als PER abgebildet werden. Dies entspricht dem System der Living Standards in HTML und JavaScript.

In den folgenden Abschnitten sollen einige der wichtigsten PSRs und PERs überblicksartig besprochen werden. Weitere Details können auf der PHP-FIG Webseite entnommen werden.

5.3.2 PSR-1

PSR-1 ist der *Basic Coding Standard*. Er enthält einfache Richtlinien, wie PHP-Code formatiert werden soll um in Komponenten und Frameworks einheitlich zu erscheinen. Folgende Richtlinien zählen dazu:

- **PHP Tags:** PHP-Code darf nur in den langen `<?php ?>` Tags oder den kurzen Echo-Tags `<?= ?>` stehen. Andere Tags sind nicht zulässig.
- **Zeichenkodierung:** PHP-Dateien müssen UTF-8 kodiert sein. Die Dateien dürfen keine Byte Order Mark (BOM) enthalten.
- **Nebeneffekte:** Eine PHP-Datei soll *entweder* Dinge definieren (wie Klassen, Funktionen, Konstanten etc.) *oder* Logik mit Nebeneffekten ausführen, jedoch niemals beides. Als Nebeneffekte werden etwa das Einbinden von externen Dateien mittels `require`, das Erzeugen von Output (mittels `echo`) und dergleichen bezeichnet.
- **Autoloading:** Alles Namespaces und Klassen müssen einer Autoloading PSR (PSR-4 oder auch PSR-0, siehe dazu Abschnitt 5.3.4) folgen, d. h. jede Klasse

¹¹<https://www.php-fig.org/psr/>

¹²<https://www.php-fig.org/per/>

muss sich in einer eigenen Datei befinden und einen Namespace mit mindestens einer Ebene (einen Vendor-Namespace) aufweisen.

- **Klassennamen:** Klassennamen werden im StudlyCaps (auch Upper Camel Case, Studly Case oder Pascal Case) Format geschrieben, z. B. `MyClassName`.
- **Konstantennamen:** Konstanten werden komplett in Großbuchstaben geschrieben, Underscores werden zur Worttrennung verwendet, z. B. `SUM` oder mit Underscores `NR_OF_ELEMENTS`.
- **Methodennamen:** Methoden werden im Camel Case notiert, bei dem der erste Buchstabe klein geschrieben wird, z. B. `calculateTotalSum()`.
- **Variablennamen:** PSR-1 überlässt die Formatierung von Variablennamen explizit den Autor*innen. Es können dabei `$StudlyCaps`, `$camelCase` oder auch `$under_score` zum Einsatz kommen, jedoch immer konsistent.

5.3.3 PER Coding Style

Aufbauend auf PSR-1 setzt die *PER Coding Style* auf striktere Regeln und deren Einhaltung. Dieser Coding-Stil erweitert und ersetzt seit Juli 2022 PSR-12 (welche wiederum den Vorläufer PSR-2 ersetzte) und wurde wie dieser aus vielen bekannten Frameworks übernommen bzw. erstellt und stellt somit einen verbreiteten Konsens dar. Im Gegensatz zu den statischen PSRs handelt es sich hierbei um einen *Living Standard*. Aktuell ist Version 3.0.0 vom 14. Juli 2025. Folgende Regeln gehören dazu:

- **PSR-1 implementieren:** Der Code folgt den Regeln aus PSR-1.
- **Dateien und Zeilenumbrüche:** Alle Dateien enthalten nur Linux-Zeilenumbrüche (LF), die letzte Zeile darf nicht leer sein. Das schließende PHP-Tag `?>` muss in Dateien, die nur PHP-Code beinhalten, entfallen. Es gibt keine harte Obergrenze für die Zeilenlänge; die weiche Grenze beträgt 120 Zeichen, empfohlen werden maximal 80 Zeichen pro Zeile. Überflüssige Whitespace-Zeichen müssen entfernt werden. Leerzeilen für bessere Lesbarkeit sind erlaubt, jede Zeile darf nur eine PHP-Anweisung beinhalten.
- **Einrückungen:** Code darf ausschließlich mit 4 Leerzeichen (nicht mit Tabulatoren) eingerückt werden.
- **Abschließendes Komma (Trailing Commas):** Erstreckt sich eine Liste über mehrere Zeilen, muss nach dem letzten Element ein abschließendes Komma gesetzt werden. Dies gilt zwingend für Arrays, Parameterlisten in Funktionsdefinitionen, Argumentlisten bei Funktionsaufrufen, `use`-Listen in Closures und Zweige in `match`-Ausdrücken. In einzeiligen Listen darf kein abschließendes Komma stehen.
- **Schlüsselwörter und Typen:** PHP-Schlüsselwörter und Typen (`int`, `string`, `true`, `null` etc.) werden ausschließlich klein geschrieben. Es ist die Kurzform der Typbezeichner zu verwenden (`bool` statt `boolean`, `int` statt `integer`). Bei Union Types (`int|string`) und Intersection Types (`A&B`) dürfen keine Leerzeichen um die Trenner gesetzt werden; ist `null` Bestandteil eines Union Types, so steht es an letzter Stelle bzw. wird bei einem einzelnen Typ als `?T` abgekürzt.
- **Reihenfolge von Deklarationen:** Die Reihenfolge am Dateianfang ist strikt

vorgegeben: `<?php`, Datei-Docblock, `declare`-Statements, `namespace`, `use`-Statements (für Klassen, Funktionen, Konstanten in separaten Blöcken).

- **Klassen, Traits und Interfaces:** Die öffnende Klammer steht in einer eigenen Zeile. Sichtbarkeitsmodifikatoren sind für alle Eigenschaften, Konstanten und Methoden zwingend. Bei der *Constructor Property Promotion* werden Sichtbarkeit und Typ direkt im Konstruktor angegeben. Bei der Instanziierung müssen die runden Klammern auch ohne Argumente gesetzt werden (`new Foo()`).
- **Reihenfolge der Modifikatoren:** Wenn mehrere Modifikatoren auftreten, ist deren Reihenfolge fest vorgeschrieben: `abstract/final`, Sichtbarkeit (`public`, `protected`, `private`), Set-Sichtbarkeit (z. B. `private(set)`), `static`, `readonly`, Typdeklaration, Name.
- **Methoden und Funktionsaufrufe:** Die öffnende Klammer folgt dem Namen ohne Leerzeichen. Erstrecken sich die Argumente eines Aufrufs über mehrere Zeilen, müssen sie jeweils um eine Ebene eingerückt werden; die schließende Klammer muss dann in einer eigenen Zeile (zusammen mit der schließenden Klammer der Anweisung, falls vorhanden) stehen. Bei benannten Argumenten steht kein Leerzeichen vor dem Doppelpunkt, jedoch eines danach.
- **Kontrollstrukturen:** Ein Leerzeichen nach dem Schlüsselwort (`if`, `switch` etc.) ist zwingend. Die öffnende Klammer steht in derselben Zeile. `else` und `elseif` stehen in derselben Zeile wie die schließende Klammer des vorangegangenen Blocks. Der Körper jeder Struktur muss in geschweiften Klammern eingeschlossen sein.
- **Operatoren:** Binäre Operatoren (Arithmetik, Vergleich, Zuweisung, logisch etc.) sowie der ternäre Operator müssen von je einem Leerzeichen umgeben sein. Inkrement-/Dekrement-Operatoren stehen ohne Whitespace direkt am Operanden, Type Casts ohne Leerzeichen innerhalb der Klammern (`(int) $x`). Wird ein Ausdruck umgebrochen, muss der Operator am Beginn der neuen Zeile stehen.
- **Arrays:** Arrays müssen mit der kurzen Syntax (`[]`) deklariert werden. Bei mehrzeiligen Arrays steht die öffnende Klammer in derselben Zeile wie das Gleichheitszeichen, jeder Wert auf einer eigenen Zeile, mit abschließendem Komma nach dem letzten Element.
- **Closures und Arrow Functions:** Closures werden mit einem Leerzeichen nach `function` deklariert. Bei Arrow Functions darf hingegen kein Leerzeichen zwischen `fn` und der öffnenden Klammer stehen: `fn(int $x) => $x * 2;`.
- **Enumerations:** Die Fälle (`case`) werden in PascalCase geschrieben. Enums können `public` Konstanten enthalten, unterstützen aber keine Vererbung; daher müssen nicht-öffentliche Methoden `private` (statt `protected`) sein.
- **Property Hooks:** Mit PHP 8.4 eingeführte `get`- und `set`-Hooks folgen einer eigenen Formatierungsregel: Mehrzeilige Hook-Bodys werden wie Methodenkörper eingerückt; besteht der Hook aus einem einzigen Ausdruck, kann er mit `=>` verkürzt werden (z. B. `get => $this->value;`).
- **Heredoc und Nowdoc:** Der schließende Identifikator darf eingerückt werden, muss jedoch auf derselben Ebene wie der restliche Code innerhalb des Blocks stehen.
- **Attribute:** Attribute (z. B. `#[Override]`) müssen in einer eigenen Zeile direkt vor

der Struktur (Klasse, Methode, Eigenschaft) stehen. Werden sie bei Parametern genutzt, stehen sie direkt vor dem jeweiligen Parameter in derselben oder einer eigenen Zeile.

5.3.4 PSR-4 und PSR-0

PSR-4 und PSR-0 sind sogenannte Autoloading-Strategien und beschreiben, wie Klassen und Namespaces mit der Platzierung im Dateisystem korrelieren.

Eine erste, lange gültige Fassung war PSR-0, die besagte, dass jeder Namespace als eigenes Verzeichnis zu existieren hatte. An den Klassennamen wurde schließlich noch `.php` gehängt und so konnten sehr einfach Klassen gefunden und geladen werden.

Die Klasse `Acme\Logging\AdvancedLogging` würde im Verzeichnis `lib\vendor` des Projekts vom Autoloader unter `lib\vendor\Acme\Logging\AdvancedLogging.php` gesucht. Die Klasse `Fhooe\MTD\WebTechnologies` entsprechend unter `lib\vendor\Fhooe\MTD\WebTechnologies.php`.

Diese Strategie ist jedoch unflexibel, da sie – je nach Verschachtelungstiefe der Namespaces – viele Verzeichnisse erzeugt. Außerdem muss immer vom selben Grundverzeichnis – in diesem Fall `lib\vendor` – ausgegangen werden. Aus diesem Grund wurde mit PSR-4 eine flexiblere Autoloading-Strategie entworfen, die nun statt PSR-0 eingesetzt wird.

Im Unterschied zu PSR-0 kann bei PSR-4 ein Basisverzeichnis definiert werden, was die Verzeichnisstruktur weniger komplex gestaltet. So kann festgelegt werden, dass Klassen des `Acme` Namespaces im Verzeichnis `src` liegen. Somit liegt dann die Klasse `Acme\Logging\AdvancedLogging` dann in `src\Logging\AdvancedLogging.php`. Alternativ kann auch `Acme\Logging` auf `src` gemapped werden, dann ergibt sich folgende Verzeichnisstruktur: `src\AdvancedLogging.php`.

In der in Abschnitt 5.2.6 gezeigten Datei `composer.json` ist etwa ersichtlich, dass Klassen mit dem Namespace `Fhooe\Router` direkt im Verzeichnis `src` liegen. D. h., es gibt keine Unterordner-Struktur `Fhooe\Router`, da dies nur unnötige Komplexität mit sich bringen würde.

PSR-4 gibt auch genau vor, wie der Autoloader selbst implementiert werden muss. Da jedoch Composer einen PSR-4 kompatiblen Autoloader generiert (siehe Abschnitt 5.2.4), gibt es eigentlich keinen Grund, dies selbst zu tun.

5.3.5 Weitere PSRs

Neben den erwähnten PSRs gib es noch eine Vielzahl weiterer, von denen hier nur sechs oberflächlich erwähnt werden sollen. Genaue Details und weitere PSRs finden sich auf der PHP-FIG-Webseite.

- **PSR-3 Logger Interface:** Definiert ein standardisiertes Interface für Klassen, die Ereignisse loggen. Dies macht es möglich, dass Logger-Komponenten einfach ausgetauscht werden können.
- **PSR-5 PHPDoc Standard:** Beschreibt wie die Dokumentation von PHP-Code aussehen soll (derzeit noch ein Entwurf).
- **PSR-6 Caching Interface:** Beschreibt ein Interface für Komponenten, die Caching unterstützen.

- **PSR-7 HTTP Message Interface:** Beschreibt Interfaces für HTTP-Messages. Die Komponente Guzzlehttp etwa implementiert dieses Interface für Requests und Responses.
- **PSR-18 HTTP Client:** Ein gemeinsames Interface zum Versenden und Empfangen von HTTP Requests und Responses. Komponenten wie Guzzlehttp können so vereinheitlichte Interfaces nützen und sind austauschbar.
- **PSR-20 Clock:** Beschreibt ein Interface, mit dem die Systemzeit ausgelesen werden kann.

5.3.6 Standards einhalten

Nachdem nun über die Wichtigkeit von Standards geschrieben wurde, bleibt noch die Frage, wie diese einfach eingehalten werden können. Entwickler*innen können unmöglich immer alle Details im Kopf behalten, weshalb es der Unterstützung von Werkzeugen bedarf, um den Standards zu folgen. Dies betrifft in erster Linie PSR-1 und PER Coding Style, da sich diese mit der Formatierung von Code auseinandersetzen.

IDEs konfigurieren

Der einfachste Weg, bereits beim Schreiben von Code den PSR/PER-Richtlinien zu folgen ist, die verwendete IDE (sofern möglich) bereits auf PSR-1 und PER-CS zu konfigurieren.

PhpStorm etwa bietet unter `Settings >> Editor >> Code Style >> PHP` die Möglichkeit, den Code-Stil nach dem PER Coding Style 2.0.0 zu setzen. Dazu wird unter `Set from...` die Option `PER-CS 2.0.0` ausgewählt.

Diese Option setzt die Platzierung von Klammern, Leerzeichen und einigen anderen Dingen auf die von PER-CS 2.0.0 vorgegebenen Regeln. Abbildung 5.5 zeigt einen Screenshot aus der Konfiguration von PhpStorm. PER-CS 3.0.0 ist zur Zeit noch nicht verfügbar, jedoch bereits als fehlend gemeldet¹³.

Überprüfung mittels Tools: PHP_CodeSniffer

Während die korrekte Konfiguration einer IDE hilft, grundsätzliche Probleme beim Editieren zu vermeiden, so können sich dennoch Fehler einschleichen, die subtiler sind, oder auch durch die automatischen Editierhilfen nicht immer abgefangen werden. Typische Beispiele sind etwa der korrekte Zeilenumbruch, der unter Windows nicht LF, sondern CRLF lautet, eine fehlende Leerzeile am Ende oder auch Zeilenlängen, die 120 Zeichen übersteigen.

Solche Fehler können im Nachhinein jedoch leicht mit einem Tool überprüft und auch korrigiert werden, dem *PHP_CodeSniffer*¹⁴ – kurz **phpcs**. Es kann als PHAR-Datei oder global über Composer installiert werden und wird über die Kommandozeile aufgerufen. Im **fhooe-web-dock** Docker Container ist **phpcs** bereits installiert und kann ohne Installation verwendet werden.

Die Syntax ist denkbar einfach:

¹³<https://youtrack.jetbrains.com/issue/WI-81988/Implement-PER-3-codestyle-in-PHP-formatters>

¹⁴https://github.com/PHP-CSStandards/PHP_CodeSniffer

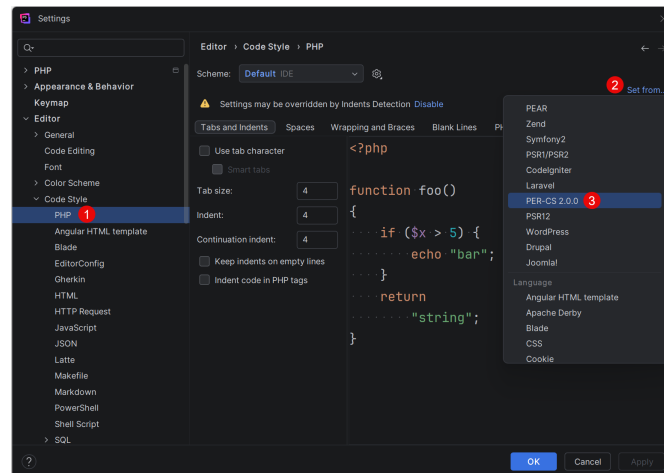


Abbildung 5.5: Der Code-Stil von PhpStorm kann nach den Regeln von PER 2.0.0 konfiguriert werden.

```
1 $ phpcs /path/to/code/myfile.php
```

Alternativ kann auch nur ein Verzeichnis angegeben werden, was alle Dateien innerhalb überprüft. Standardmäßig verwendet `phpcs` jedoch den PEAR Coding-Standard. Um PSR-12 (PER Coding Style ist noch nicht verfügbar) zu verwenden, kann ein optionaler Parameter angegeben werden:

```
1 $ phpcs --standard=PSR12 /path/to/code/myfile.php
```

Nach der Ausführung generiert `phpcs` einen Fehlerreport.

Im Falle des Chuck Norris-Beispiels werden zur Simulation von Problemen in einer neuen Datei Einrückungen verschoben, ein Trailing Comma entfernt und die öffnende geschwungene Klammer der if-Bedingung in die nächste Zeile verschoben. Danach wird das Tool wie folgt aufgerufen und generiert diesen Output:

```
1 $ phpcs --standard=PSR12 index_phpcerrors.php
```

```
FILE: /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcerrors.php
```

```
-----
FOUND 2 ERRORS AFFECTING 2 LINES
```

```
-----
37 | ERROR | [x] Expected 1 space after closing parenthesis; found newline
53 | ERROR | [x] Line indented incorrectly; expected at least 4 spaces, found 2
```

```
-----
PHPCBF CAN FIX THE 2 MARKED SNIFF VIOLATIONS AUTOMATICALLY
-----
```

```
Time: 75ms; Memory: 8MB
```

Gefunden wurden in diesem Fall zwei Fehler:

- In Zeile 37 sollte die geschwungene Klammer nach einem Leerzeichen zur vorigen Klammer stehen, gefunden wurde jedoch ein Zeilenumbruch.
- In Zeile 53 ist der Code um zwei Leerzeichen zu wenig eingerückt.

Das Trailing Comma wird nicht gefunden, da dies in PSR-12 noch nicht enthalten war. Die restlichen Fehler können nun händisch korrigiert werden oder mit Hilfe eines zweiten Tools, dem *PHP Code Beautifier and Fixer* – kurz **phpcbf** – automatisch korrigiert werden. In der Regel wird **phpcs** im Aufruf einfach durch **phpcbf** ersetzt:

```
1 phpcbf --standard=PSR12 index_phpcserrors.php
```

Die führt zur Korrektur der Fehler mit folgendem Output:

PHPCBF RESULT SUMMARY

FILE	FIXED	REMAINING
/.../wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php	3	0

A TOTAL OF 2 ERRORS WERE FIXED IN 1 FILE

Time: 103ms; Memory: 8MB

Ein erneutes Aufrufen von **phpcs** zeigt, dass keine Fehler mehr vorhanden sind.

PHP_CodeSniffer kann auch in viele IDEs, wie z. B. PhpStorm automatisch integriert werden. Unter **Settings** » **Editor** » **Inspections** » **PHP** » **Quality tools** kann ein Haken bei der Einstellung **PHP_CodeSniffer validation** gesetzt werden. Voraussetzung ist, dass dieser zuvor korrekt unter **Settings** » **PHP** » **Quality Tools** » **PHP_CodeSniffer** konfiguriert wurde. Die automatische Inspektion führt **phpcs** nach jedem Speichervorgang aus und zeigt Fehler bereits in PhpStorm an.

Alternative: PHP-CS-Fixer

Eine weit verbreitete Alternative zu PHP_CodeSniffer ist der *PHP CS Fixer*¹⁵ – kurz **php-cs-fixer**. Im Gegensatz zum Gespann **phpcs**/**phpcbf** vereint er die Analyse und die automatische Korrektur in einem einzigen Werkzeug. Vor allem aber unterstützt er den aktuellen *PER Coding Style 3.0* bereits direkt als eingebautes Regelset und ist damit – anders als **phpcs** – auf dem aktuellen Stand der PHP-FIG-Spezifikation.

Wie **phpcs** kann auch **php-cs-fixer** als PHAR-Datei oder über Composer (Paket **friendsofphp/php-cs-fixer**) installiert werden und ist im **fhooe-web-dock** Docker Container bereits vorhanden. Der zentrale Befehl ist **fix**, dem als Argument eine Datei oder ein Verzeichnis übergeben wird. Standardmäßig verwendet **php-cs-fixer** jedoch das **@PSR12**-Regelset; um stattdessen den *PER Coding Style 3.0* anzuwenden, wird das Regelset **@PER-CS3x0** über die Option **--rules** angegeben:

```
1 php-cs-fixer fix --rules=@PER-CS3x0 /path/to/code/myfile.php
```

Ein wesentlicher Unterschied zu **phpcs** besteht darin, dass **fix** die Dateien unmittelbar verändert – das Tool ist also primär als Fixer und nicht als Linter konzipiert. Soll **php-cs-fixer** – analog zu **phpcs** – lediglich auflisten, welche Verstöße er finden würde, ohne die Dateien zu modifizieren, kann der Befehl **check** verwendet werden, eine Kurzform für **fix --dry-run**. Mit der zusätzlichen Option **--diff** werden die konkreten Änderungen im **udiff**-Format ausgegeben:

¹⁵ <https://github.com/PHP-CS-Fixer/PHP-CS-Fixer>

```
1 php-cs-fixer check --rules=@PER-CS3x0 --diff index_phpcserrors.php
```

Auf das gleiche manipulierte Chuck Norris-Beispiel angewendet, generiert das Tool folgenden Output:

```
1) index_phpcserrors.php
----- begin diff -----
--- /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php
+++ /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php
@@ -28,14 +28,13 @@
foreach ($categories as $category) {
echo "<option value=\"\$category\">{$category}</option>";
}
-      ?>
+?>
</select>
<button type="submit">Roundhouse!</button>
</form>

<?php
-if (isset($_POST["category"]))
-{
+if (isset($_POST["category"])) {
echo "<h2>Random fact from category {$_POST["category"]}</h2>";

$factResponse = $client->request(
@@ -43,7 +42,7 @@
"https://api.chucknorris.io/jokes/random",
[
"query" => [
-      "category" => $_POST["category"]
+      "category" => $_POST["category"],
],
],
);

----- end diff -----
```

```
Found 1 of 1 files that can be fixed in 0.018 seconds, 19.41 MB memory used
```

Die Verstöße entsprechen inhaltlich jenen, die zuvor auch **phpcs** gemeldet hat, allerdings wird auch das trailing Comma gefunden und das schließende PHP-Tag wird an den Anfang der Zeile verschoben.

Wo bei **phpcs** jedoch im Anschluss **phpcbf** aufgerufen werden müsste, genügt bei **php-cs-fixer** der Wechsel des Befehls von **check** auf **fix**, um die Korrektur tatsächlich durchzuführen:

```
1 php-cs-fixer fix --rules=@PER-CS3x0 index_phpcserrors.php
```

```
1) index_phpcserrors.php
```

```
Fixed 1 of 1 files in 0.021 seconds, 19.41 MB memory used
```

Ein erneuter **check**-Lauf bestätigt, dass keine Verstöße mehr vorhanden sind.

Während sich `phpcs` sehr gut für eine schnelle Überprüfung über die Kommandozeile eignet, ist `php-cs-fixer` stärker auf die dauerhafte Verwendung im selben Projekt ausgerichtet. In diesem Fall wird statt der Option `--rules` eine Konfigurationsdatei `.php-cs-fixer.dist.php` im Projekt-Wurzelverzeichnis abgelegt, in der Regelset und einzubeziehende Pfade festgelegt werden. Liegt eine solche Datei vor, genügt der Aufruf von `php-cs-fixer fix` ohne weitere Argumente. Eine minimale Konfiguration für PER 3.0 sieht etwa so aus:

```
1 <?php
2 return (new PhpCsFixer\Config())
3     ->setRules(['@PER-CS3x0' => true])
4     ->setFinder((new PhpCsFixer\Finder())->in(__DIR__));
```

Auch `php-cs-fixer` kann in PhpStorm integriert werden. Unter **Settings** **Editor** **Inspections** **PHP** **Quality tools** kann ein Haken bei der Einstellung **PHP CS Fixer validation** gesetzt werden, sofern das Tool zuvor unter **Settings** **PHP** **Quality Tools** **PHP CS Fixer** korrekt konfiguriert wurde. Verstöße werden dann auch nach jedem Speichervorgang direkt im Editor angezeigt. Über **Code** **Reformat Code with PHP CS Fixer** lassen sie sich unmittelbar beheben.

5.4 Weiterführende Literatur

Die beste Referenz zum Thema Komponenten und Standards bietet mit Sicherheit [7]. Es ist die modernste Referenz zu diesem Thema. Eine gute Übersicht online bietet ebenfalls [3]. Eine etwas kompaktere Einführung zum Thema Package Management bietet [5].

Quellenverzeichnis

- [1] Nils Adermann und Jordi Boggiano. *The composer.json schema - Composer*. 30. Okt. 2025. URL: <https://getcomposer.org/doc/04-schema.md> (siehe S. 5-11).
- [2] Paul M. Jones. *A History Of PHP Frameworks/Library Collections*. 15. Dez. 2022. URL: <https://github.com/pmjones/php-history> (besucht am 03.05.2026) (siehe S. 5-1).
- [3] Josh Lockhart und Phil Sturgeon. *PHP: The Right Way*. 22. Jan. 2026. URL: <https://phptherightway.com/> (besucht am 14.04.2026) (siehe S. 5-21).
- [4] Packagist. *Install Statistics*. 3. Mai 2026. URL: <https://packagist.org/statistics> (besucht am 03.05.2026) (siehe S. 5-5).
- [5] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 5-21).
- [6] Jamie York. *Awesome PHP. A curated list of amazingly awesome PHP libraries, resources and shiny things*. 23. Apr. 2026. URL: <https://github.com/ziadoz/awesome-php> (besucht am 02.05.2026) (siehe S. 5-5).
- [7] Matt Zandstra. *PHP 8 Objects, Patterns, and Practice: Volume 2. Mastering Essential Development Tools*. 7. Aufl. New York, USA: Apress, 22. Juli 2025 (siehe S. 5-21).